syai-claude-workshop

advanced Claude Code workshop — hooks, subagents, commands

90 minutes · hands-on · type along

Welcome

advanced Claude Code, hands-on

What we cover

- **Hooks** — scripts that fire on Claude Code lifecycle events
- **Subagents & the Agent SDK** — spawn focused workers; invoke from your own code
- **Slash commands, Skills, Plugins** — make Claude Code yours

What we skip today: MCP, plan mode internals, alternative agent CLIs.

Setup verify

Run these now — every chapter writes to `.claude/`.

```
# Verify your Claude Code install
claude --version
# expect: 1.x.x or 2.x.x

# Verify project-local Claude config dir exists
mkdir -p .claude
ls -la .claude
# expect: directory listed, possibly empty
```

✓ Check: `claude --version` prints a semver string and `.claude/` exists in your current dir. If either fails, fix before the next slide — every chapter writes into `.claude/`.

How to follow

- Type the commands. Don’t watch.
- Each chapter ends with a `✓ Check:` — if yours doesn’t match, ask before we move on.
- Slides stay live at the workshop URL — re-walk at your own pace afterward.

Ready

Three topics, ninety minutes, hands on keyboard.

Next: **Hooks**.

Hooks

scripts that fire on Claude Code’s lifecycle

Why hooks

- **Guardrails.** Block tool calls that touch what you don’t want touched.
- **Telemetry.** Log every tool call, every prompt, every session start.
- **Glue.** Run a script when Claude finishes a turn — re-run tests, format, deploy.

Hooks are how you make Claude Code react to itself.

Lifecycle events

Event	Fires when
`SessionStart`	a new Claude Code session begins
`UserPromptSubmit`	you press enter on a prompt
`PreToolUse`	before Claude executes a tool call
`PostToolUse`	after a tool call completes
`Stop`	Claude finishes a turn

Each hook is a shell command. Output and exit code matter — block by exiting non-zero from `PreToolUse`.

settings.json shape

Hooks live in `~/.claude/settings.json` (user-wide) or `.claude/settings.json` (project).

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "*",
        "hooks": [
          { "type": "command", "command": "/path/to/script.sh" }
        ]
      }
    ]
  }
}
```

`matcher` filters by tool name (e.g. `"Edit"`, `"Bash"`, `"*"` for all). `command` runs in your shell with hook context piped on stdin.

Before — no hook

You ask Claude to clean up old logs.

```
> claude
You: clean up everything in logs/, it's cluttered
Claude: I'll remove the contents of logs/.
  → Bash(rm -rf logs/*)
  → exit 0
You: ...wait. did that include the deploy logs from yesterday?
Claude: yes, those are gone.
```

No hook. Claude reaches for the dangerous primitive, and it runs. There is nothing between Claude’s tool call and the kernel.

The hook

```
#!/usr/bin/env bash
# ~/.claude/hooks/pre-tool-use.sh — block rm -rf and .env reads
payload=$(cat)
tool=$(echo "$payload" | jq -r '.tool_name')
cmd=$(echo "$payload" | jq -r '.tool_input.command // .tool_input.file_path // ""')

if [[ "$tool" == "Bash" && "$cmd" =~ rm[[:space:]]+(-[a-z]*r[a-z]*f|--recursive) ]]; then
  echo '{"decision":"block","reason":"rm -rf blocked by hook"}' >&2
  exit 2
fi

if [[ "$cmd" =~ \.env(\b|/) && "$cmd" != *.env.sample ]]; then
  echo '{"decision":"block","reason":".env access blocked"}' >&2
  exit 2
fi
```

Wiring it in `.claude/settings.json`

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash|Read|Edit|Write",
        "hooks": [
          { "type": "command", "command": "$HOME/.claude/hooks/pre-tool-use.sh" }
        ]
      }
    ]
  }
}
```

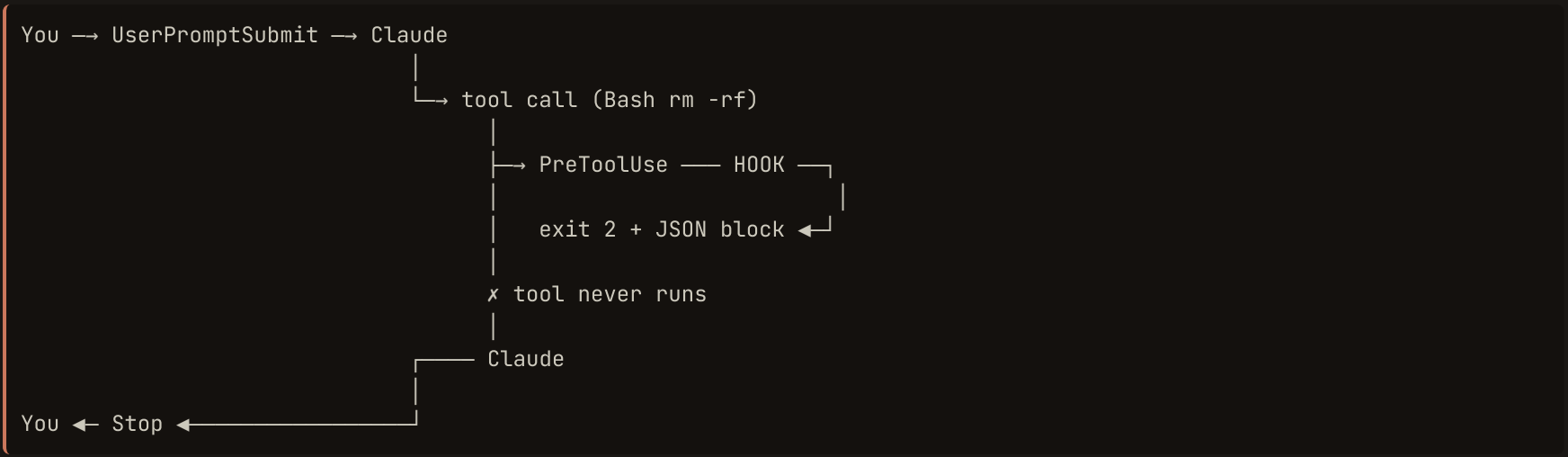
`matcher` filters by tool name (pipe-separated). The hook receives the full tool call JSON on stdin. **Hook config loads at session start** — start a fresh `claude` session for it to take effect.

After — hook wired

```
> claude
You: clean up everything in logs/, it's cluttered
Claude: I'll remove the contents of logs/.
  → Bash(rm -rf logs/*)
  ✗ BLOCKED: {"decision":"block","reason":"rm -rf blocked by hook"}
Claude: that was blocked by your PreToolUse hook.
        Want me to list what's in logs/ first instead?
You: yes
  → Bash(ls -la logs/)
  → exit 0
```

Same prompt, same Claude. The hook intercepted before the kernel saw `rm -rf`.

What just happened



The hook sits between Claude's intent and the tool's execution. Exit `0` — call proceeds. Exit `2` with `decision:"block"` — tool never runs and Claude sees the reason.

One hook, two guardrails

The same `pre-tool-use.sh` already blocks `.env` access (slide 6, second `if`).

```
You: read our .env so we can debug the deploy
Claude: → Read(.env)
  ✗ BLOCKED: {"decision":"block","reason":".env access blocked"}
```

Allowing `.env.sample` keeps the example-config workflow open. Defensive hooks compose — one matcher, many rules in the script.

When hooks don’t fire

- **New session?** Hook config loads at session start. Old sessions don’t pick it up.
- **Executable?** `chmod +x ~/.claude/hooks/pre-tool-use.sh`.
- **Matcher right?** `Bash|Read|Edit|Write`, not `*` (which doesn’t match anything for hooks).
- **Output going where?** Hooks run silently by default. While debugging, append `>> /tmp/hook.log 2>&1` to the command in `settings.json`.
- **Path expansion?** Use `$HOME`, not `~` — JSON doesn’t expand `~`.

Exercise (5 min)

Wire a `PostToolUse` hook that appends one line per tool call to `~/.claude/tool-log.txt`. You write the script and the JSON.

```
# Hint: a 4-line shell script is enough.
# stdin is JSON; append timestamp + tool_name to the log.
```

1. Create `~/.claude/hooks/log-tool.sh` — read stdin, write `$(date -u +%FT%TZ) $(jq -r .tool_name)` to the log.
2. Add a `PostToolUse` entry to `~/.claude/settings.json` with `matcher: ""` (matches all tools).
3. Start a fresh `claude` session. Ask it to list files.

✓ Check: `wc -l ~/.claude/tool-log.txt` shows ≥1; `tail -1` shows a timestamp + a tool name like `Bash` or `Read`.

Hooks recap

- 5 lifecycle events covered today: `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`, `Stop`. (13 events exist total.)
- `PreToolUse` exit `2` + `{"decision":"block"}` is your guardrail.
- Config in `~/.claude/settings.json` (user) or `.claude/settings.json` (project).
- Hook config loads at session start; debug with `>> /tmp/hook.log 2>&1`.

Further reading: disler/claude-code-hooks-mastery — all 13 events, payload shapes, TTS integrations.

Next: subagents — when one Claude isn't enough.

Subagents & the Agent SDK

spawn focused workers; invoke from your own code

Why subagents

- **Independent context.** A subagent doesn’t pollute the parent’s context window with research it did.
- **Parallelism.** Three subagents can investigate three branches of a problem at once.
- **Specialization.** Different system prompts, different tool sets, different focus.

When you find yourself thinking "I need a fresh Claude for this" — that’s a subagent.

The Task tool

Inside Claude Code, **Claude itself can spawn subagents** by calling the `Task` tool.

You don’t call Task. Claude does — when its system prompt or your prompt says it should.

```
parent Claude → Task → fresh subagent → returns final message → parent continues
```

The subagent has its own context, its own tool budget, and its own system prompt (from the agent definition file).

`.claude/agents/<name>.md`

A subagent is a markdown file with frontmatter. The frontmatter declares the agent's identity; the body becomes its system prompt.

```
---
name: code-reviewer
description: Reviews diffs for bugs, style, and missed edge cases.
tools: [Read, Grep, Bash]
---

You are a strict but fair code reviewer.
Read the diff. Flag concrete issues with file:line references.
Skip style nits unless they materially affect readability.
```

`tools` is the allowlist — the subagent only sees these. Omit to inherit the parent's tool set.

Two ways in

- **Claude-invoked.** You ask the parent to do something; it decides a subagent is the right move and calls Task with the agent name.
- **User-invoked.** You explicitly say *"use the code-reviewer agent on this diff"* — the parent forwards.

Same primitive, different trigger. The agent file doesn't know or care which path called it.

Live: list what’s there

```
# Many CC installs ship with example agents. List yours:
ls -la .claude/agents/ 2>/dev/null
ls -la ~/.claude/agents/ 2>/dev/null

# Project-local agents in .claude/agents/ override
# user-wide agents in ~/.claude/agents/.
```

```
✓ Check: at least one of those directories exists.
If not, you'll create one in the exercise.
```

Live: anatomy of a real agent

```
# Pick an existing agent and read it.
# If you don't have one, skip to the exercise.
ls ~/.claude/agents/ | head -1
# example output: planner.md

cat ~/.claude/agents/planner.md | head -20
```

Notice the shape: frontmatter (`name` , `description` , optional `tools`) + a body that reads as a system prompt. That’s the entire contract.

Exercise (8 min) — write `code-reviewer`

Step 1. Create `.claude/agents/code-reviewer.md` in your project:

```
---
name: code-reviewer
description: Reviews recent changes for bugs, style, and edge cases.
tools: [Read, Grep, Bash]
---

You are a strict but fair code reviewer.
When invoked, run `git diff` to see recent changes.
Flag concrete issues with file:line references.
Skip style nits unless they materially affect readability.
End with a one-line verdict: APPROVE or REQUEST CHANGES.
```

Step 2. In Claude Code, ask:

> *"Use the code-reviewer agent on the most recent commit."*

```
✓ Check: Claude calls the Task tool with agent "code-reviewer"
and the review ends with APPROVE or REQUEST CHANGES.
```

The SDK bridge

The Agent SDK exposes the same primitive — query a Claude agent — from your own code. Same model, same tool semantics, no CLI wrapper.

`@anthropic-ai/claude-agent-sdk` on npm. TypeScript-first. Streams.

Use it when the invocation is programmatic: a cron job, a CI hook, your own app.

Live: minimal SDK call

```
import { query } from "@anthropic-ai/claude-agent-sdk";

const result = query({
  prompt: "List the markdown files in this repo and summarize each.",
});

for await (const message of result) {
  if (message.type === "result") {
    console.log(message.result);
  }
}
```

That’s the whole "spawn a Claude" surface.

Which one when

- **CLI / inside Claude Code.** Interactive. You're at a terminal, riffing.
- **SDK.** Programmatic. Cron job, CI hook, your own app. No terminal in the loop.

The agent definition file format is the same on both sides. Write it once, invoke from either.

Subagents recap

- Subagents = fresh Claude with its own context, system prompt, tool set.
- Invoke from inside CC via the Task tool (Claude- or user-triggered).
- Define in `.claude/agents/<name>.md` with `name`, `description`, optional `tools`.
- Same primitive lives in `@anthropic-ai/claude-agent-sdk` for programmatic use.

Next: how to package your own commands and skills.

Commands · Skills · Plugins

three primitives for making Claude Code yours

Why these three

- **Slash commands.** A reusable prompt with a name. Type `/standup` instead of writing the prompt.
- **Skills.** Progressive disclosure — Claude reads a skill's index, fetches details only when relevant.
- **Plugins.** A bundle of commands + skills + agents you can share or install.

Together: write once, invoke fast, share with the team.

The pyramid

```
plugin (bundle)
   │
   ├── slash commands  → fast, on-demand prompts
   ├── skills          → discoverable knowledge, on-demand depth
   └── agents          → focused workers (previous chapter)
```

A plugin is just packaging. Underneath, every entry is one of the primitives you already know.

`.claude/commands/<name>.md`

A slash command is a markdown file. Type `/<name>` in Claude Code; the file's body becomes the prompt.

```
---
description: Daily standup summary from recent commits.
---

Read the last 24 hours of git activity in this repo.
For each meaningful commit, write one line:
"<short-sha>: <one-sentence summary of what changed>".
End with three bullets: in progress, blockers, next.
Keep it under 15 lines total.
```

`description` shows up in the `/` menu autocomplete. The body is the prompt — just markdown.

Arguments

Pass arguments after the slash command name:

```
/standup --since 3d
```

The full string after the command name is appended to the prompt body. Reference it in your prompt with natural language ("use the arguments provided") or with the placeholder below.

Live: list your commands

```
# Project-local commands live in .claude/commands/
ls .claude/commands/ 2>/dev/null || echo "(none yet)"

# User-wide commands live in ~/.claude/commands/
ls ~/.claude/commands/ 2>/dev/null || echo "(none yet)"
```

✓ Check: one or both directories listed. Even if empty, the command ran without error — the path is valid. Commands in `~/.claude/commands/` are available in every project.

Exercise (5 min) — write `/standup`

1. Create `.claude/commands/standup.md`:

```
---
description: Daily standup summary from recent commits.
---

Run `git log --since="24 hours ago" --oneline` in this repo.
For each commit, write one line:
"<short-sha>: <one-sentence summary of what changed>".
Then three bullets labeled: 'in progress', 'blockers', 'next'.
Keep it under 15 lines total.
```

2. In Claude Code, type `/standup` and press Enter.

3. ✓ Check: Claude runs `git log --since="24 hours ago" --oneline`, then prints one line per commit and three labeled bullets, all under ~15 lines.

`.claude/skills/<name>/SKILL.md`

A skill is a directory with a `SKILL.md` index. Claude sees the index — short and descriptive — and only reads supporting files when relevant.

```
.claude/skills/
└── postgres-migrations/
    ├── SKILL.md           ← always-loaded index
    ├── examples/up-down.sql
    └── playbooks/safe-migration.md
```

The `SKILL.md` says *what* the skill is for and *what files are inside*. Claude pulls the rest only when the situation calls for it.

Progressive disclosure

The win: large knowledge bases don’t blow up your context.

- **Always loaded:** every `SKILL.md` index in scope (a few hundred tokens each).
- **Loaded on demand:** the rest of the skill’s files, only when Claude decides they’re relevant.

You write the index to be a good table of contents. The skill’s depth lives in the supporting files.

Live: skills you already have

```
# You almost certainly have skills installed already.
ls ~/.claude/skills/ 2>/dev/null | head -10

# Pick one and read its index:
FIRST=$(ls ~/.claude/skills/ 2>/dev/null | head -1)
cat ~/.claude/skills/"$FIRST"/SKILL.md 2>/dev/null | head -30
```

✓ Check: you see a `SKILL.md` index with a short description and a list of supporting files or subdirectories. That's the contract — index = always loaded, the rest = on demand.

Plugins — distribution layer

A plugin bundles **commands + skills + agents** so you can install or share them as a unit.

```
my-plugin/
├── commands/
│   └── standup.md
├── skills/
│   └── git-conventions/
│       └── SKILL.md
└── agents/
    └── code-reviewer.md
```

Install once, get every primitive inside. We won't build one in 25 minutes — the pattern is just your `.claude/` directory, packaged.

Which primitive when

- **Slash command** — short reusable prompt. Body fits on one screen. Invoke by name.
- **Skill** — body wouldn’t fit on one screen. You want progressive disclosure; Claude pulls depth only when needed.
- **Agent** — needs its own context, its own tool budget, its own focused instruction set.

Sliding scale, same family. Pick the smallest one that does the job.

Commands · Skills · Plugins recap

- Slash commands = `.claude/commands/<name>.md`. Body is the prompt. Frontmatter `description` powers the `/` menu.
- Skills = `.claude/skills/<name>/SKILL.md` index plus on-demand support files. Progressive disclosure keeps context tight.
- Plugins = a directory bundling all three for distribution. Not built today — the structure is the concept.

Same backbone, different scope. Pick the smallest one that does the job.

Recap

three primitives, real composability

What you can do now

- **Hooks** — make Claude Code react to its own lifecycle. Block, log, redirect.
- **Subagents** — spawn focused workers with their own context. Invoke from inside CC or from the SDK.
- **Commands · Skills · Plugins** — package your own primitives, distribute, share.

The pattern: Claude Code is programmable. Treat it like one.

Where to go next

- Hooks: `~/.claude/settings.json` reference — see `claude --help` and the official docs site.
- Subagents: `.claude/agents/` directory in any project; `@anthropic-ai/claude-agent-sdk` on npm.
- Skills: your own `~/.claude/skills/` already has examples — `ls ~/.claude/skills/`.
- Further reading on hooks: disler/claude-code-hooks-mastery — lifecycle taxonomy, PreToolUse patterns.

This deck stays live at the workshop URL — revisit any chapter.

Questions

Ask now or open an issue on the workshop repo.

Thanks for typing along.